

FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Wed 7 FEBRUARY 2018 - Part Theory

LAST + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): Syntax and Semantics of Languages
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

1 Regular Expressions and Finite Automata 20%

1. Consider the following right uni-linear grammar G , over the two-letter terminal alphabet $\Sigma = \{ a, b \}$ and the three-letter nonterminal alphabet $V = \{ S, T, W \}$ (axiom S), which generates a regular language $L = L(G)$:

$$G \left\{ \begin{array}{lcl} S & \rightarrow & a T \\ T & \rightarrow & b T \mid a W \mid a \\ W & \rightarrow & a T \mid \varepsilon \end{array} \right.$$

Answer the following questions (use the space on the next pages):

- (a) By using a systematic method, transform the grammar G above into a finite-state automaton A_1 (not necessarily deterministic) that accepts language L .
 - (b) By using the method of the linear language equations, from grammar G obtain a regular expression R_1 that generates language L .
 - (c) By using the *BMC* method (node elimination), obtain a regular expression R_2 from the automaton A_1 of point (a) above. Check that the regular expression R_2 is equivalent to the regular expression R_1 of point (b).
 - (d) By using the *BS* method (Berry-Sethi), derive a deterministic finite-state automaton A_2 that accepts language L . Check that the automaton A_2 is equivalent to the automaton A_1 of point (a).
 - (e) Design a deterministic automaton A_3 that accepts the complement language $\overline{L}$ of language L .
 - (f) (optional) Determine whether the language L is local or not, and justify your answer. If the language L is not local, in a systematic way design a finite-state automaton A_4 that accepts the local language that has the same local sets as language L .
-

2 Free Grammars and Pushdown Automata 20%

1. Consider the context-free language L below, over the two-letter alphabet $\Sigma = \{ a, b \}$:

$$L = \{ a^h b^k \mid h > k \geq 1 \wedge h \text{ is even} \wedge k \text{ is odd} \}$$

Answer the following questions:

- (a) Write the four shortest strings of language L . Write a *BNF* grammar G , no matter if ambiguous, that generates language L . Draw a syntax tree for the sample valid string:

$$a^6 b^3$$

- (b) Argue whether the grammar G written at point (a) is ambiguous or not. In the case that grammar G is ambiguous, write an equivalent non-ambiguous grammar G' and justify it.
- (c) (optional) Consider the complement language $\overline{L}$ of language L , with respect to the alphabet Σ :

$$\overline{L} = \Sigma^* - L$$

Is the complement language $\overline{L}$ context-free or not ? Explain your answer.

2. Consider the fragment of a programming language that includes statements of these four kinds: *assignment*, *iterative*, *conditional* and *branch*. This fragmentary language is informally specified as follows:

- A phrase of the language is a non-empty list of statements as said above, each of them followed by a semicolon “;”.
- An *assignment* statement consists of a variable name, an assignment operator “=” and an expression.
- In the entire grammar, all the variables and expressions (or conditions) must be represented by the terminals **var** and **exp**, respectively, without expansion.
- There are three types of *iterative* statement. The first (and simplest) type consists of a non-empty statement list enclosed by the keywords **do** and **end**. The second type adds to the first one an initial clause with a keyword **while** followed by an iteration condition. The third type adds to the first one a final clause with a keyword **when** followed by a termination condition. The clauses **while** and **when** may not be simultaneously present in the same iterative statement.
- There are two types of *conditional* statement. The first type is the customary two-way conditional **if-then-else-endif**, where both ways **then** and **else** contain a non-empty statement list, and the **else** way is optional. The second type is a non-empty linear list of clauses **if-do**, each of them containing only one statement, and the whole list is enclosed by the two keywords **cond** and **endcond**.
- There are two types of *branch* statement, which just consist of the keywords **terminate** and **exit**. The statements **terminate** and **exit** may only occur inside an iterative statement and a conditional statement, respectively.
- All the statement types described above can be freely nested, except those constrained by the previous item.

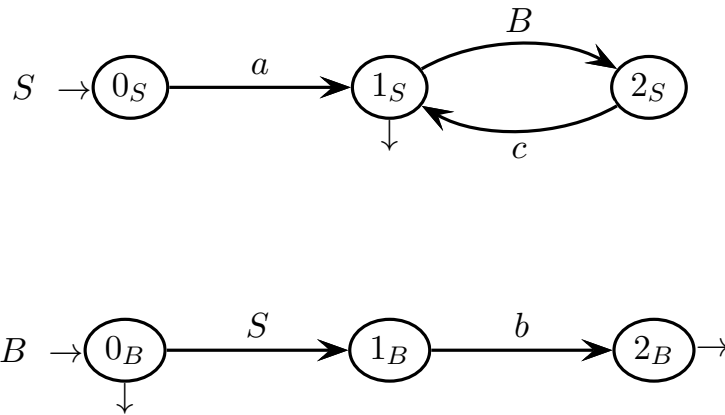
For any unspecified minor detail, see the example below, and you may also follow the usual conventions, e.g., those of the C language. Example:

```
var = exp;
do
  cond
    if exp do exit;
    if exp do
      while exp do
        if exp then terminate; else var = exp; endif;
      end;
    endcond;
  do
    var = exp;
    if exp then exit; endif;
  end when exp;
end;
```

Write a non-ambiguous grammar, in general of type *EBNF*, that models the fragmentary language described above.

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the following grammar G , represented as a machine net over the three-letter terminal alphabet $\Sigma = \{ a, b, c \}$ and the two-letter nonterminal alphabet $V = \{ S, B \}$ (axiom S):



Answer the following questions (use the figures / tables / spaces on the next pages):

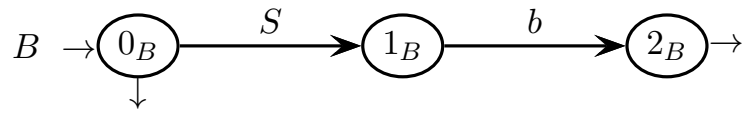
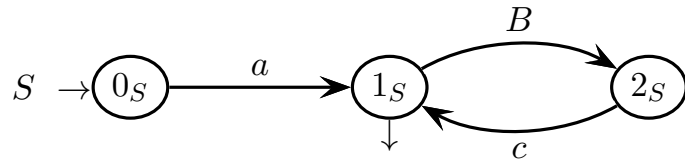
- (a) Draw the complete pilot of grammar G and prove that grammar G is of type $ELR(1)$. Explain your answer.
- (b) Write all the guide sets on the shift arcs, call arcs and exit arrows of the net of grammar G , and say if grammar G is of type $ELL(1)$. Explain your answer.
- (c) Nonterminal B is nullable. Modify the machines of nonterminals S and B , but without changing the number of states of each machine and the generated language $L(G)$, so that nonterminal B becomes non-nullable. Is the resulting net still of type $ELR(1)$? Explain your answer.
- (d) (optional) Simulate the bottom-up analysis of the following valid sample string:

$$a c a b c \in L(G)$$

Show the piecewise construction of the syntax tree of the string in the analysis.

please draw here the complete pilot – question (a)

please add here the call arcs and all the guide sets – question (b)



please answer here – question (c)



simulation table for the *ELR* analysis to be completed - question (d) - (the number of rows is not significant)

[illegible]

draw here the piecewise tree construction

4 Language Translation and Semantic Analysis 20%

1. A (possibly empty) binary string has an *even* parity (respectively an *odd* parity) if it includes an *even* (respectively *odd*) number of bits 1. For instance, strings ε , 00 and 0101 have an even parity, whereas strings 1, 01 and 010110 have an odd one.

The source language L_s , over the three-letter alphabet $\Sigma = \{ 0, 1, s \}$, is the set of all the (possibly empty) sequences of (possibly empty) binary strings, where each binary string is followed by a symbol s . Examples:

$$01s101s \quad 110s001s00s \quad 01ss101s$$

The source language L_s is generated by the following source grammar G_s (axiom S):

$$G_s \begin{cases} S \rightarrow B s S \mid \varepsilon \\ B \rightarrow 0 B \mid 1 B \mid \varepsilon \end{cases}$$

Please answer the following questions and keep in mind that you may change the source grammar (but not the source language), when necessary. The solutions that use simpler grammars, in terms of number of nonterminals and rules, are preferred.

- (a) Design a translation scheme (or grammar) that defines a translation function τ_1 , where:
 - each binary string included in the source string is translated into a symbol e or o if it has an even or odd parity, respectively
 - all the symbols s included in the source string are canceled

Examples:

$$\begin{aligned} \tau_1(01s101s) &= oe \\ \tau_1(110s001s00s) &= eoe \\ \tau_1(01ss101s) &= oee \end{aligned}$$

- (b) Design a translation scheme (or grammar) defining a translation function τ_2 that maps a source string into a destination string $e^h o^k$, where h and k ($h, k \geq 0$) are the numbers of binary strings with an even and odd parity, respectively, in the source string.

Examples:

$$\begin{aligned} \tau_2(01s101s) &= eo \\ \tau_2(110s001s00s) &= eeo = e^2o \\ \tau_2(01ss101s) &= eeo = e^2o \end{aligned}$$

- (c) (optional) Can the translation functions τ_1 and τ_2 be defined by means of a regular translation expression or be computed by a deterministic finite-state device? Justify your answer.

2. Consider the grammar below (axiom S), which generates a (possibly empty) two-level list. Each sublist consists of letters a (possibly none) and ends with one letter b .

$$\left\{ \begin{array}{l} 1: S \rightarrow X \\ 2: X \rightarrow A X \\ 3: X \rightarrow \varepsilon \\ 4: A \rightarrow a A \\ 5: A \rightarrow b \end{array} \right.$$

A sample two-level list, with three sublists, is:

$$b a b a^2 b$$

See also the syntax tree on the next pages.

The *length* ≥ 0 of a sublist is the number of elements, i.e., letters a , in the sublist.

We want to decide whether in a list the length of the sublists increases, from left to right, exactly by one, starting from an empty sublist (length 0). If this property is satisfied, a boolean attribute e assumes value *true*, else *false*. For the sample list above, the attribute e is *true*. If the whole list is empty, the attribute e is conventionally *true*.

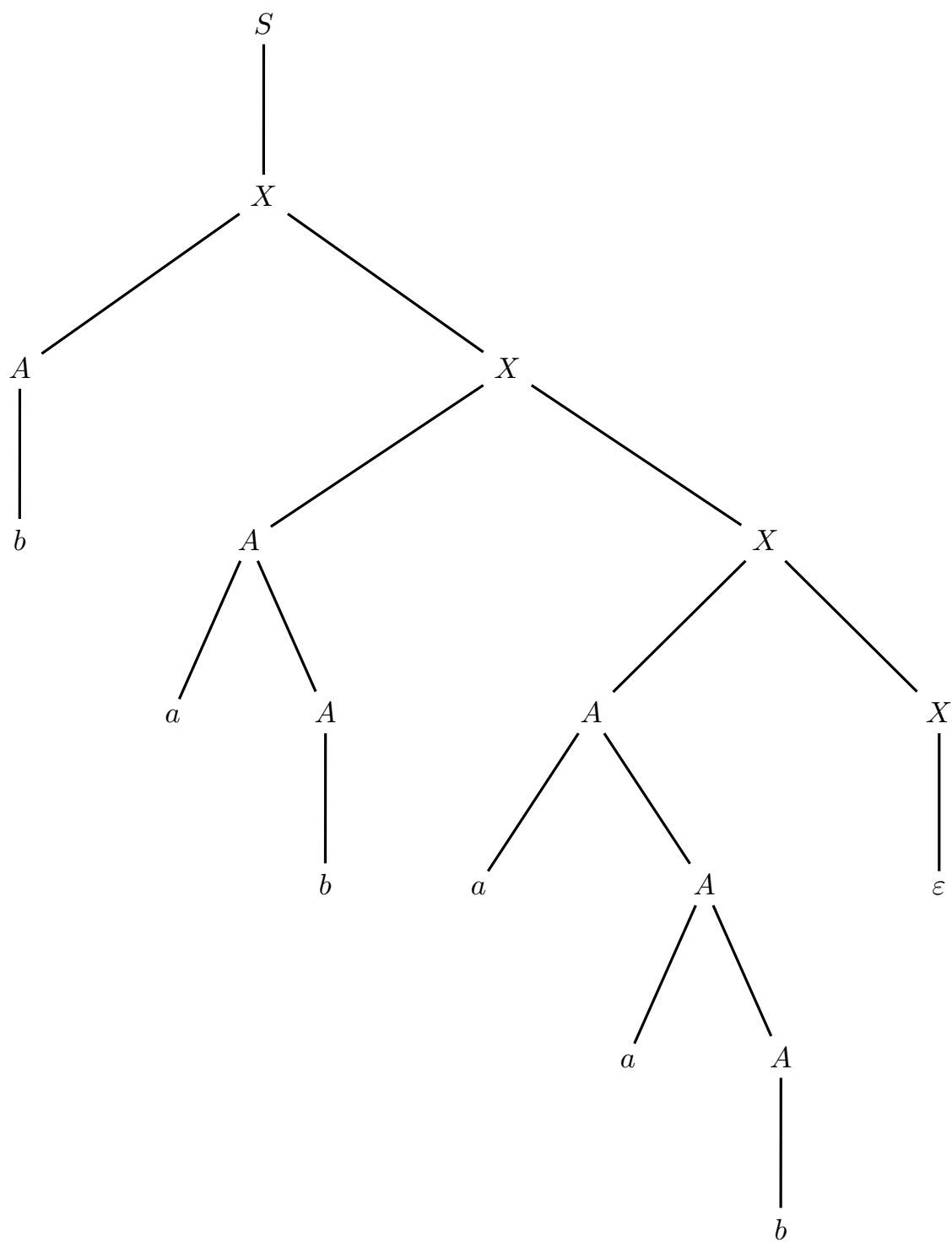
Attributes permitted to use (no other attributes are allowed):

<i>name</i>	<i>type</i>	<i>domain</i>	<i>symbol</i>	<i>meaning</i>
l	left	integer	A	length ≥ 0 of a sublist
n	right	integer	X	required length of the sublist immediately appended to a node X
e	left	boolean	X, S	<i>true</i> if the sublists appended to a node X , or to the root S , have a length that increases by one from left to right (starting from 0), else <i>false</i>

Answer the following questions (use the tables / trees / spaces on the next pages):

- Decorate the tree of the sample string $b a b a^2 b$, prepared on the next page, with the attribute values. Pay attention to the type of each attribute (left or right).
- Write an attribute grammar, based on the syntax above and using only the permitted attributes (do not change the attribute types), that computes in the tree root the correctness attribute e for the whole list. The attribute grammar must be of type one-sweep. Please argue that your grammar is so.
- (optional) Write the procedure of the semantic analyzer (which exists as the attribute grammar must be of type one-sweep) for nonterminal X . Is it possible to integrate syntactic and semantic analysis? Please explain your answer.

syntax tree to be decorated – question (a)



syntax *semantics* – question (b)

1: $S_0 \rightarrow X_1$

2: $X_0 \rightarrow A_1 X_2$

3: $X_0 \rightarrow \varepsilon$

4: $A_0 \rightarrow a A_1$

5: $A_0 \rightarrow b$

semantic procedure of nonterminal X – question (c)

